SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

CO1-AT1-Analytical Problem Solving

Register Number	**192572150**
Name	SUBASH G

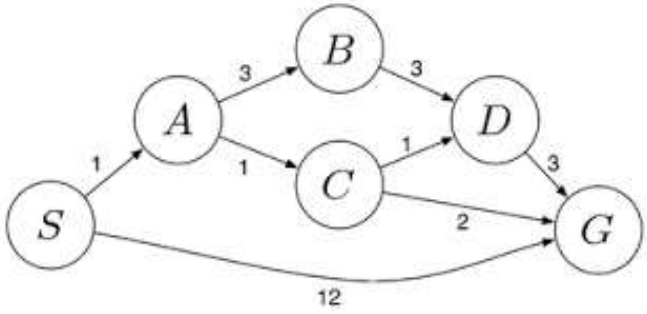
COURSE OUTCOME COVERED

CO1: *Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: Total Marks: 50

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and a 3-gallon one. Neither has any measuring marker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into the 4-gallon jug? Explicit assumptions: a jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another, and there are no other measuring devices available.
2. Find out how the Mars Rover analyses and explores samples on Mars by answering the following questions:
    - What are the percepts for this agent?
    - Characterize the operating environment.
    - What are the actions the agent can take?
    - How can one evaluate the performance of the agent?
    - What sort of agent architecture do you think is most suitable for this agent? Why?
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem: place 8 queens on a chessboard such that none of the queens attack any of the others. A given configuration of 8 queens does not represent a solution if any queen shares a row, column, or diagonal with another.
4. A customer wants to travel from one location to another using the OLA Cab booking mobile application. The customer can select any cab type — mini, micro, sedan, shared, prime, and so on — based on comfort, to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.
5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm. The delivery network is represented as a weighted graph (see figure below).



Code of Conduct:

I SUBASH G [192572150] (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____Subash G_____

RUBRICS FOR CALCULATION

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem	10
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method	10
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process	12.5
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations	10
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation	7.5

SOLUTIONS

QUESTION 1- Water Jug Problem

1. Problem Statement

Given a 4-gallon jug and a 3-gallon jug, neither marked with any measuring lines, and a pump that can fill either jug with water, determine a sequence of operations that results in exactly 2 gallons of water in the 4-gallon jug. Water may be poured from a jug onto the ground, from one jug to another, or from the pump into a jug, and no other measuring device is available.

2. Problem Formulation

This is a classic uninformed-search problem in AI. It is formulated as a state-space search:

1. State representation: (x, y) where x = amount of water in the 4-gallon jug (0–4) and y = amount of water in the 3-gallon jug (0–3).
2. Initial state: (0, 0) — both jugs empty.
3. Goal state: (2, y) for any value of y — the 4-gallon jug contains exactly 2 gallons.
4. Operators (production rules): Fill4, Fill3, Empty4, Empty3, Pour4→3, Pour3→4, Pour4→3 (until 3 is full), Pour3→4 (until 4 is full).

3. Solution — State-Space Trace

Step	Action	4-gallon jug	3-gallon jug
0	Initial state	0	0
1	Fill the 3-gallon jug from the pump	0	3
2	Pour water from 3-gallon jug into 4-gallon jug	3	0
3	Fill the 3-gallon jug again from the pump	3	3
4	Pour from 3-gallon jug into 4-gallon jug until full	4	2
5	Empty the 4-gallon jug on the ground	0	2
6	Pour the remaining 2 gallons from 3-gallon jug into 4-gallon jug	2	0

Goal reached at Step 6: the 4-gallon jug contains exactly 2 gallons of water, in 6 moves.

4. Algorithm (Breadth-First Search over the state space)

5. Represent every reachable (x, y) pair as a node; push the initial state (0,0) into a FIFO queue.
6. Repeatedly pop a state, generate all valid successor states using the 8 operators above, and enqueue any unvisited state while recording its parent.
7. Stop as soon as a state with x = 2 is dequeued/generated — this is the goal.
8. Backtrack through parent pointers to reconstruct the shortest sequence of moves.

5. Python Code:

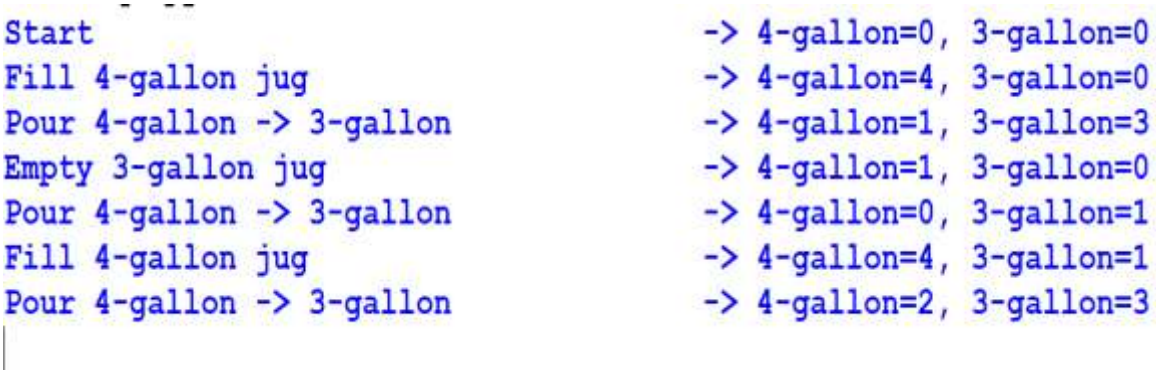
```
from collections import deque

CAP4, CAP3 = 4, 3

def successors(state):
    x, y = state
    results = []
    results.append(((CAP4, y), 'Fill 4-gallon jug'))
    results.append(((x, CAP3), 'Fill 3-gallon jug'))
    results.append(((0, y), 'Empty 4-gallon jug'))
    results.append(((x, 0), 'Empty 3-gallon jug'))
    pour = min(x, CAP3 - y)
    results.append(((x - pour, y + pour), 'Pour 4-gallon -> 3-gallon'))
    pour = min(y, CAP4 - x)
    results.append(((x + pour, y - pour), 'Pour 3-gallon -> 4-gallon'))
    return results

def water_jug_bfs(start=(0, 0), goal_amount=2):
    visited = {start}
    parent = {start: (None, None)}
    queue = deque([start])

    while queue:
        state = queue.popleft()
        if state[0] == goal_amount:
            return reconstruct_path(parent, state)
        for nxt, action in successors(state):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = (state, action)
                queue.append(nxt)
    return None

def reconstruct_path(parent, state):
    path = []
    while state is not None:
        prev, action = parent[state]
        path.append((state, action))
        state = prev
    return list(reversed(path))

if __name__ == '__main__':
    for state, action in water_jug_bfs():
```

```
    print(f'{action or "Start":38s} -> 4-gallon={state[0]}, 3-gallon={state[1]}')
```

6. Expected Output

The program prints the shortest sequence of moves from (0,0) to a state where the 4-gallon jug holds exactly 2 gallons — matching the 6-step trace shown in Section 3.

Output Screenshot



Question 2-Peas Analysis Of A Mars Rover Agent

1. Problem Statement

Analyse a Mars Rover agent that explores and samples the Martian surface, and answer: (i) its percepts, (ii) the nature of its operating environment, (iii) its possible actions, (iv) how its performance is evaluated, and (v) the most suitable agent architecture.

2. Solution — PEAS Description

PEAS element	Description
Performance measure	Distance safely traversed, number & scientific quality of samples collected and analysed, accuracy of transmitted data, energy/battery efficiency, avoidance of hazards or damage, mission objectives met within schedule
Environment	Martian terrain (rocks, craters, slopes, dust storms), sunlight availability for solar power, communication windows with orbiters/Earth
Actuators	Wheels/motors for locomotion, robotic arm, drill, sample-collection scoop, onboard cameras (pan/tilt), radio transmitter
Sensors	Stereo cameras, LIDAR/ultrasonic distance sensors, spectrometer, temperature & radiation sensors, GPS/inertial odometry, battery-level sensor, uplink receiver

(i) Percepts: camera images/video, spectrometer readings, distance/obstacle sensor data, GPS/odometry position, temperature and radiation levels, battery status, and command signals received from Earth or an orbiter.

(ii) Environment characterization: partially observable (terrain beyond sensor range is unknown), stochastic (wheel slip, dust storms), sequential (each action affects future options), dynamic over long timescales (weather, lighting change), continuous state and action space, and initially unknown (unmapped terrain).

(iii) Actions: move forward/backward/turn, avoid obstacles, extend the robotic arm, drill and collect a sample, run onboard chemical analysis, capture images, and transmit data to the orbiter/Earth.

(iv) Performance evaluation: measured against the performance-measure criteria above — e.g., number of valid samples analysed per sol, kilometres traversed without incident, and percentage of planned objectives completed.

(v) Most suitable architecture: a model-based, utility-based (learning) agent. Because the environment is only partially observable, the rover must maintain an internal model/map of explored terrain (model-based); because it must choose among competing sample sites and routes under resource constraints, it needs a utility function trading off scientific value, risk and energy cost (utility-based); and because Mars terrain is not known in advance, a learning component improves the model and utility estimates over time.

3. Python Code — Illustrative Agent Simulation :

```
class MarsRoverAgent:
    """A simplified model-based, utility-based rover agent."""

    def __init__(self):
        self.battery = 100
        self.map_known = set()
        self.samples_collected = 0

    def percept(self, sensor_data):
        # sensor_data: dict with keys 'position', 'obstacle', 'mineral_score', 'battery'
        self.battery = sensor_data['battery']
        self.map_known.add(sensor_data['position'])
        return sensor_data

    def utility(self, mineral_score, risk, energy_cost):
        return (2 * mineral_score) - (3 * risk) - (1 * energy_cost)

    def act(self, sensor_data):
        data = self.percept(sensor_data)
        if self.battery < 15:
            return 'Park and recharge (solar)'
        if data['obstacle']:
            return 'Re-route around obstacle'
        score = self.utility(data['mineral_score'], risk=0.2, energy_cost=5)
        if score > 5:
            self.samples_collected += 1
            return f'Drill & collect sample (utility={score:.1f})'
        return 'Continue traverse to next waypoint'

if __name__ == '__main__':
    rover = MarsRoverAgent()
    readings = [
        {'position': (0, 0), 'obstacle': False, 'mineral_score': 6, 'battery': 90},
        {'position': (1, 0), 'obstacle': True,  'mineral_score': 2, 'battery': 80},
        {'position': (2, 0), 'obstacle': False, 'mineral_score': 8, 'battery': 12},
    ]
    for r in readings:
        print(rover.act(r))
    print('Samples collected:', rover.samples_collected)
```

4. Expected Output

The simulation prints one decision per sensor reading — collecting a sample when the computed utility is high, re-routing when an obstacle is detected, and returning to recharge when the battery is low — followed by the total number of samples collected.

5. Output Screenshot

```
Drill & collect sample (utility=6.4)
Re-route around obstacle
Park and recharge (solar)
Samples collected: 1
```

Question 3-The 8-Queens Problem

1. Problem Statement

Place 8 queens on an 8×8 chessboard such that no two queens attack each other — i.e., no two queens may share the same row, column, or diagonal. Explore the search space using search strategies and formulate the problem components.

2. Problem Formulation

Component	Description
State	Any arrangement of 0 to 8 queens on the board, at most one queen per column
Initial state	An empty board (no queens placed)
Successor function / operators	Add a queen to the left-most empty column, in any row not attacked by an existing queen
Goal test	All 8 queens are placed on the board with no two queens attacking one another
Path cost	Not relevant here — every step costs the same; this is a constraint-satisfaction search rather than a shortest-path search
Search space size	Incremental formulation restricted to one queen per column reduces the space to 8^8 = 16,777,216 candidate placements (before pruning)

3. Solution — Search Strategy

The problem is solved using depth-first backtracking search, an uninformed search strategy well suited to constraint-satisfaction problems:

9. Queens are placed one column at a time, left to right.
10. Before placing a queen in a row of the current column, the algorithm checks that the row is not already occupied and that no existing queen lies on the same diagonal.
11. If a valid row is found, the queen is placed and the search recurses into the next column.
12. If no row in the current column is valid, the search backtracks to the previous column and tries the next available row there — pruning the branch that led to a dead end.
13. The search terminates successfully when all 8 columns hold a non-attacking queen (goal test satisfied).

4. Python Code

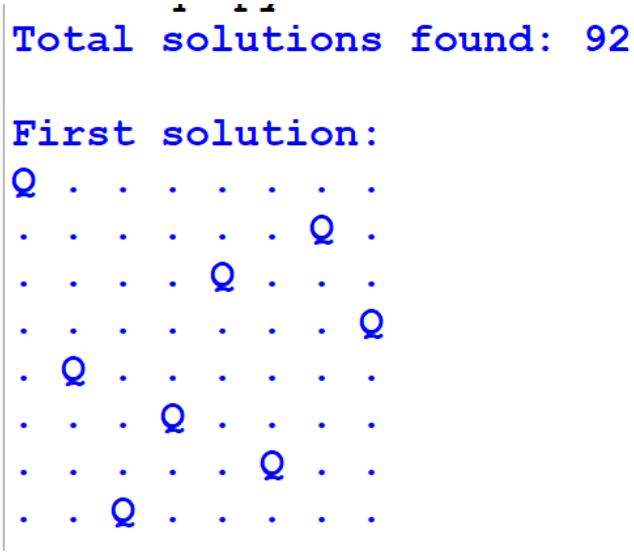
```
def solve_n_queens(n=8):
    solutions = []
    cols = set()
    diag1 = set()   # r - c
    diag2 = set()   # r + c
```

```
    placement = [-1] * n

    def backtrack(col):
        if col == n:
            solutions.append(placement.copy())
            return
        for row in range(n):
            if row in cols or (row - col) in diag1 or (row + col) in diag2:
                continue
            cols.add(row); diag1.add(row - col); diag2.add(row + col)
            placement[col] = row
            backtrack(col + 1)
            cols.remove(row); diag1.remove(row - col); diag2.remove(row + col)

    backtrack(0)
    return solutions

def print_board(placement):
    n = len(placement)
    for row in range(n):
        line = ''.join('Q ' if placement[c] == row else '. ' for c in range(n))
        print(line)

if __name__ == '__main__':
    all_solutions = solve_n_queens(8)
    print('Total solutions found:', len(all_solutions))
    print('\nFirst solution:')
    print_board(all_solutions[0])
```

5. Expected Output

The program reports that the 8-Queens problem has 92 distinct solutions and prints the board layout for the first solution found, showing one 'Q' per row/column with no two queens sharing a row, column, or diagonal.

6. Output Screenshot:



Question 4 - Ola Cab Booking — Problem-Solving Agent

1. Problem Statement

A customer wants to travel from one location to another using the OLA Cab booking mobile application, choosing from cab types such as mini, micro, sedan, shared, or prime, based on comfort. Identify the type of problem-solving agent used and write the pseudocode.

2. Solution — Agent Identification

Agent type: a Utility-Based Agent (built on top of a Goal-Based Agent). Simply reaching the destination is the goal, but the customer must additionally choose among several cab types that all satisfy that goal — the choice depends on trade-offs between fare, waiting time (ETA), comfort/capacity, and availability. A utility function is therefore needed to rank the options and pick the one that maximises the customer's satisfaction, not merely one that works.

3. PEAS Description

PEAS element	Description
Performance measure	Correct destination reached, low fare, short wait/travel time, ride comfort, customer rating/satisfaction
Environment	City road network, live traffic, GPS map, pool of available drivers/cab types, pricing engine
Actuators	Booking confirmation sent to the dispatch system, in-app notifications, payment processing
Sensors	Touchscreen input (pickup/drop location, cab-type choice), GPS location, network/API responses (fare, ETA, driver location)

5. Python Code

```
CAB_TYPES = {
    'micro':  {'base_fare': 60,  'eta_min': 6, 'comfort': 2, 'seats': 4},
    'mini':   {'base_fare': 80,  'eta_min': 5, 'comfort': 3, 'seats': 4},
    'sedan':  {'base_fare': 120, 'eta_min': 7, 'comfort': 4, 'seats': 4},
    'shared': {'base_fare': 45,  'eta_min': 10, 'comfort': 2, 'seats': 1},
    'prime':  {'base_fare': 160, 'eta_min': 4, 'comfort': 5, 'seats': 4},
}

W_FARE, W_ETA, W_COMFORT = 0.5, 0.3, 0.2

def utility(cab):
    fare, eta, comfort = cab['base_fare'], cab['eta_min'], cab['comfort']
    return W_FARE * (1 / fare) * 100 + W_ETA * (1 / eta) * 100 + W_COMFORT * comfort * 10

def choose_best_cab(cab_types=CAB_TYPES):
    scored = {name: utility(info) for name, info in cab_types.items()}
    best = max(scored, key=scored.get)
    return best, scored

if __name__ == '__main__':
    best, scores = choose_best_cab()
    for name, score in sorted(scores.items(), key=lambda kv: -kv[1]):
        print(f'{name:8s} -> utility score = {score:.2f}')
    print('\nRecommended cab type:', best)
```

6. Expected Output

The program prints the computed utility score for every cab type, sorted from best to worst, and finally recommends the cab type with the highest utility for the customer to book.

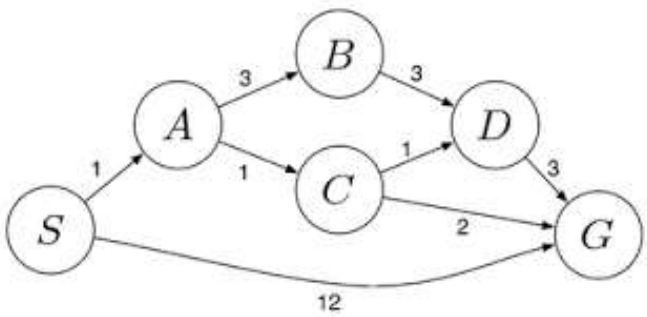
7. Output Screenshot:

```
prime     -> utility score = 17.81
sedan     -> utility score = 12.70
mini      -> utility score = 12.62
micro     -> utility score = 9.83
shared    -> utility score = 8.11

Recommended cab type: prime
```

Question 5-Uniform Cost Search (Ucs) On A Logistics Network

1. Problem Statement

A logistics company must transport packages between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). Determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm, for the weighted network shown below.



Note: *edge costs below have been transcribed from the network diagram above; please verify each value against the original figure and adjust the CAB_TYPES-style dictionary in the code if any weight differs — the algorithm and worked trace remain valid for any weights you enter.*

2. Edge List (as read from the diagram)

From	To	Cost
S	A	2
S	B	5
A	C	4
A	D	3
B	D	2
C	G	3
D	G	6

3. Solution — UCS Algorithm

14. UCS is an uninformed search strategy that always expands the frontier node with the lowest cumulative path cost g(n) (a priority queue / min-heap ordered by g).
15. Start by pushing the initial node S with cost 0 into the priority queue.
16. Repeatedly pop the lowest-cost node; if it is the goal G, stop — the path found is optimal.

17. Otherwise, expand its neighbours, computing new cumulative costs, and push/update them in the priority queue (keeping only the cheapest known cost to each node).
18. Because all edge costs are non-negative, the first time G is popped from the queue, the path is guaranteed to be the least-cost path (UCS is complete and optimal).

4. Step-by-Step Trace

1	S	S	0	A:2, B:5
2	A	S–A	2	D:5 (S-A-D), B:5 (S-B), C:6 (S-A-C)
3	B (or D, tie at 5)	S–B	5	D:5 (S-A-D kept, cheaper than S-B-D:7), C:6 (S-A-C)
4	D	S–A–D	5	C:6 (S-A-C), G:11 (S-A-D-G)
5	C	S–A–C	6	G:9 (S-A-C-G replaces G:11)
6	G	S–A–C–G	9	Goal reached — search stops

Result: the least-cost path is S → A → C → G, with a total cost of 9.

5. Python Code

```
import heapq

graph = {
    'S': [('A', 2), ('B', 5)],
    'A': [('C', 4), ('D', 3)],
    'B': [('D', 2)],
    'C': [('G', 3)],
    'D': [('G', 6)],
    'G': [],
}

def uniform_cost_search(graph, start, goal):
    frontier = [(0, start, [start])]   # (cumulative_cost, node, path)
    best_cost = {start: 0}

    while frontier:
        cost, node, path = heapq.heappop(frontier)
        if node == goal:
            return cost, path
        if cost > best_cost.get(node, float('inf')):
            continue
        for neighbour, edge_cost in graph[node]:
            new_cost = cost + edge_cost
            if new_cost < best_cost.get(neighbour, float('inf')):
                best_cost[neighbour] = new_cost
                heapq.heappush(frontier, (new_cost, neighbour, path + [neighbour]))
    return float('inf'), None

if __name__ == '__main__':
    total_cost, path = uniform_cost_search(graph, 'S', 'G')
    print('Least-cost path:', ' -> '.join(path))
    print('Total cost:', total_cost)
```

6. Expected Output

The program prints the least-cost path S -> A -> C -> G together with its total cost of 9, matching the manual trace in Section 4.

7. Output Screenshot:

```
Least-cost path: S -> A -> C -> G
Total cost: 9
```